ДОКУМЕНТАЦИЯ ПРОДУКТА "NODE2CODE"

Апрель 2026

Содержание

1. О ПРОДУКТЕ 3
1.1. Что это такое 3
1.2. Для кого это 3
1.3. Ключевые особенности 4
2. СОСТАВ ПРОДУКТА 5
2.1. Плагин для Unity 5
2.2. Обучающий сайт 5
2.3. Серверная часть 6
3. ВОЗМОЖНОСТИ ПЛАГИНА 7
3.1. Типы нод 7
3.2. Как работает парсер (превращение кода в ноды) 9
3.3. Как работает генератор (превращение нод в код) 10
3.4. Как работает выполнение графов 11
3.5. Работа с файлами 11
4. ОБУЧАЮЩИЙ КУРС 12
4.1. Структура курса 12
4.2. Форматы уроков 15
4.3. Отслеживание прогресса 15
5. УСТАНОВКА ПЛАГИНА 16
6. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ 17
6.1. Создание первого графа 17
6.2. Парсинг кода в граф 17
6.3. Прохождение курса 17
7. ТЕХНИЧЕСКИЕ ТРЕБОВАНИЯ 18
8. ЗАКЛЮЧЕНИЕ 19

1. О ПРОДУКТЕ

1.1. Что это такое

Node2Code - это инструмент для Unity, который позволяет создавать игровую логику двумя способами: через визуальные ноды (блок-схемы) и через написание кода на C#. Главная особенность в том, что эти два способа можно смешивать и конвертировать одно в другое.

Пользователь может:

- Нарисовать логику в виде нод и получить готовый C# скрипт
- Написать код на C# и увидеть его в виде понятной блок-схемы
- Выполнять созданную логику прямо в игре
- Сохранять и загружать графы в JSON файлы

Продукт создан для тех, кто только начинает изучать Unity и C#, а также для опытных разработчиков, которые хотят быстро прототипировать механики.

1.2. Для кого это

- Начинающие разработчики игр, которые хотят понять, как работает код
- Геймдизайнеры, которым нужно быстро проверить игровую механику
- Студенты и преподаватели игровых направлений
- Инди-разработчики, совмещающие визуальную логику и код

1.3. Ключевые особенности

Низкий порог входа. Для работы с Node2Code не нужно знать сложный синтаксис C#. Достаточно понимать базовые вещи: переменные, условия, циклы, действия.

Наглядность. Сложная логика, написанная в коде, превращается в понятную схему. Это помогает разобраться в чужом коде или вспомнить свою собственную логику через полгода.

Двусторонность. Можно начать с визуальных блоков, а когда станет тесно - перейти к коду. И наоборот: написать сложный алгоритм в коде, а потом вывести его в ноды для презентации или обучения.

Работа в реальном времени. Созданную логику можно сразу запустить в игре, не выходя из редактора.

Сохранение. Графы сохраняются в JSON, их удобно передавать коллегам, хранить в git и версионировать.

2. СОСТАВ ПРОДУКТА

Продукт состоит из трёх частей:

2.1. Плагин для Unity

Это основная часть. Плагин встраивается в Unity и добавляет в меню Tools пункт для открытия редактора нод.

Что внутри:

- Редактор, где можно рисовать графы из нод
- Более 30 типов нод: числа, текст, математика, сравнения, условия, циклы
- Парсер, который превращает C# код в граф
- Генератор, который превращает граф в C# код
- Система выполнения, которая запускает граф прямо в игре

2.2. Обучающий сайт

Сайт, где пользователь может узнать о продукте и пройти обучение.

Что внутри:

- Описание возможностей плагина
- Инструкция по установке
- Полный курс из 8 модулей и 24 уроков
- Личный кабинет с прогрессом обучения
- Тёмная и светлая тема оформления

2.3. Серверная часть

Бэкенд, который хранит данные пользователей.

Что внутри:

- Регистрация и вход пользователей
- Хранение прогресса обучения
- API для связи сайта с базой данных

3. ВОЗМОЖНОСТИ ПЛАГИНА

3.1. Типы нод

Все ноды разделены на группы:

Литералы:

- Целые числа (int)
- Дробные числа (float)
- Правда/Ложь (bool)
- Текст (string)

Математика:

- Сложение (+)
- Вычитание (-)
- Умножение (*)
- Деление (/)
- Остаток от деления (%)
- Модуль
- Max
- Min

Сравнения:

- Равно (==)
- Не равно (!=)
- Больше (>)
- Меньше (<)
- Больше или равно (>=)
- Меньше или равно (<=)

Логика:

- И (&&)
- ИЛИ (||)
- НЕ (!)

Управление потоком:

- If (условие)
- Else (иначе)
- For (цикл со счётчиком)
- While (цикл с условием)
- Console.WriteLine (вывод в консоль)

Конвертация:

- int.Parse - текст в число
- float.Parse - текст в дробное число
- ToString - что угодно в текст

3.2. Как работает парсер (превращение кода в ноды)

Парсер читает C# код и превращает его в граф.

Что он понимает:

- Объявление переменных: int x = 10;
- Присваивание: x = 20;
- Составные операции: x += 5;
- Инкремент и декремент: x++; ++x; x--; --x;
- Арифметику: + - * / %
- Сравнения: == != > < >= <=
- Логику: && || !
- Условия: if / else
- Циклы: for, while
- Вывод в консоль: Console.WriteLine()
- Преобразование типов: int.Parse(), float.Parse(), ToString()

Парсер специально сделан так, чтобы пользователь мог писать код без лишних сложностей. Не нужно объявлять класс и метод - код оборачивается автоматически. Можно просто писать инструкции подряд, как в школьной тетради.

Что парсер НЕ понимает (пока что):

- Массивы и списки
- Сложные классы и структуры
- События и делегаты
- LINQ запросы

- Switch оператор

- Do-while цикл

- Foreach цикл

Это не баг, а осознанное ограничение. Продукт создан для обучения и быстрого прототипирования, а не для замены полноценной разработки на C#.

3.3. Как работает генератор (превращение нод в код)

Генератор обходит граф, начиная с корневых нод (тех, у которых нет входа по потоку выполнения), и последовательно генерирует C# код.

Он умеет:

- Объявлять переменные с правильными типами

- Генерировать выражения любой вложенности

- Создавать условия if/else с правильными блоками

- Генерировать циклы for и while

- Обрабатывать присваивания и операции

- Сохранять исходные выражения через ExpressionOverride (чтобы при обратном парсинге не потерялась красота кода)

3.4. Как работает выполнение графов

Граф можно выполнить прямо в Unity, без генерации кода.

Система выполнения:

- Определяет стартовые ноды (у которых нет входа execIn)
- Выполняет ноды в правильном порядке
- Передаёт значения по связям между нодами
- Обрабатывает ветвления (true/false у условий)
- Поддерживает вложенные графы (для условий и тел циклов)
- Хранит переменные в контексте выполнения

3.5. Работа с файлами

Пользователь может:

- Создать новый граф
- Сохранить граф в JSON файл
- Загрузить граф из JSON файла
- Экспортировать граф в C# скрипт

При сохранении сохраняются не только логические связи, но и позиции нод на холсте.

4. ОБУЧАЮЩИЙ КУРС

4.1. Структура курса

Курс состоит из 8 модулей и 24 уроков. Каждый модуль имеет цель, несколько уроков и практические задания.

- Модуль 1: Первые шаги в Unity

Цель: Снять страх перед интерфейсом движка и подготовить рабочую среду.

Уроки:

- Установка Unity Hub и редактора

- Обзор интерфейса Unity (Scene, Hierarchy, Inspector, Project)

- Знакомство с плагином Node2Code

Практика: Тест на знание окон Unity, создание сцены с кубом, сферой и светом.

- Модуль 2: Как мыслит Unity

Цель: Понять жизненный цикл скрипта и концепцию хранения данных.

Уроки:

- Что такое скрипт и методы Start/Update

- Создание переменной (свойства скорости)

- Передача данных в движение

Практика: Тест на понимание нодового дерева, создание скрипта с выводом в консоль.

- Модуль 3: Общение объектов

Цель: Научить объекты видеть друг друга и взаимодействовать.

Уроки:

- Поиск объектов по тегам и ссылкам

- Передача данных между объектами

Практика: Создание кнопки и двери с реакцией на касание.

- Модуль 4: Создаём игру Space Invaders

Цель: Собрать полноценный проект на визуальном программировании.

Уроки:

- Игрок и управление

- Механика стрельбы

- Враги и их логика

- Подсчёт очков и победа

Практика: Поиск ошибки в стрельбе, добавление укрытий.

- Модуль 5: От нод к C#

Цель: Показать, что ноды - это просто красивое представление кода.

Уроки:

- Анатомия ноды (что соответствует каким конструкциям C#)

- Базовый синтаксис C#

Практика: Сопоставить картинки нод со строками кода.

- Модуль 6: Практика с C# в Unity

Цель: Научить расширять плагин через код.

Уроки:

- Когда визуальных блоков недостаточно

- Пишем логику на C# (зигзагообразное движение)

Практика: Написать ноду "Удвоитель" на C#.

- Модуль 7: Из кода обратно в ноды

Цель: Показать, как плагин разбирает написанный код.

Уроки:

- Пишем скрипт в IDE

- Компиляция в дерево нод

- Анализ графа

Практика: Выбрать правильную схему для условия if, написать скрипт телепортации.

- Модуль 8: Выпускной

Цель: Дать вектор для самостоятельного развития.

Уроки:

- Итоги пройденного пути

- Где применять плагин в профессиональной разработке

Практика: Составить личный план развития.

4.2. Форматы уроков

Каждый урок имеет:

- Видео (встраивается из Rutube)
- Теорию (текстовое объяснение)
- Конспект (краткая выжимка)
- Задание (для практических уроков) при необходимости

Практические задания проверяются через встроенный редактор кода на сайте. Пользователь пишет ответ, нажимает "Проверить", и получает обратную связь.

4.3. Отслеживание прогресса

После завершения урока пользователь нажимает "Следующий урок", и прогресс сохраняется на сервере. В личном кабинете отображается:

- Сколько уроков пройдено
- Процент завершения курса
- Детальный список уроков с отметками

5. УСТАНОВКА ПЛАГИНА

1. Скачайте файл Node2Code.unitypackage с сайта
2. Откройте ваш проект в Unity
3. Нажмите Assets -> Import Package -> Custom Package
4. Выберите скачанный файл
5. Нажмите Import
6. Откройте: Window → Package Manager
7. Нажмите кнопку + (плюс) в левом верхнем углу
8. Выберите Add package from git URL...
9. Вставьте ссылку:

https://github.com/Warwlock/NodeGraphProcessor.git

10. Нажмите Add
11. В Package Manager снова нажмите +
12. Выберите Add package by name...
13. Введите:

com.unity.nuget.newtonsoft-json

14. Нажмите Add
15. Откройте плагин: в меню Unity выберите Tools → Visual Scripting

6. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

6.1. Создание первого графа

1. Откройте Tools -> Visual Scripting
2. Нажмите "Создать новый граф"
3. Через ПКМ -> Create Node -> «Тип ноды» -> «Нода» создайте ноды
4. Соедините порты нод, перетаскивая мышью от одного порта к другому
5. Нажмите "Сгенерировать код" - получите C# скрипт
6. Нажмите "Выполнить" - граф запустится прямо в окне редактора

6.2. Парсинг кода в граф

1. Откройте Tools -> Visual Scripting
2. В левой панели введите код C#
3. Нажмите "Парсить код"
4. Справа появится визуальный граф, соответствующий коду

6.3. Прохождение курса

1. На странице курса выберите модуль
2. Внутри модуля выберите урок
3. Посмотрите видео (или прочитайте теорию)
4. Для практических уроков выполните задание во встроенном редакторе кода, если они существуют у урока
5. Нажмите "Проверить" - получите обратную связь
6. Нажмите "Следующий урок" - прогресс сохранится

7. ТЕХНИЧЕСКИЕ ТРЕБОВАНИЯ

- Unity 6 (6000.0) LTS или новее
- Windows, Mac или Linux

8. ЗАКЛЮЧЕНИЕ

Node2Code - это инструмент, который делает обучение Unity и C# наглядным и понятным. Он подходит для полных новичков, которые боятся кода, и для опытных разработчиков, которые хотят быстро прототипировать механики.

Двусторонняя конвертация между нодами и кодом позволяет выбрать тот способ работы, который удобен в данный момент. Можно начать с визуальных блоков, а когда станет тесно - перейти к коду. И наоборот: написать сложный алгоритм в коде, а потом показать его в виде понятной схемы.

Встроенный курс из 24 уроков проведёт пользователя от установки Unity до написания собственных скриптов и понимания архитектуры компонентов. Все уроки содержат видео, теорию и практические задания.

Продукт создан командой из пяти человек, каждый из которых внёс свой вклад в разработку, дизайн, тестирование и документацию.